

Antigravity Document Operating System

The Engineering Standard for Autonomous Document Operations Across Agent Harnesses

1. Executive Architecture Overview

Modern LLM agents frequently fail when interacting with real-world documents. Common failure modes include destructive in-place overwrites, stripping live spreadsheet formulas into dead text, destroying multi-column layouts in Word documents, and overflowing PowerPoint slide boundaries. The **Antigravity Document Operating System (Document OS)** replaces impromptu code hallucination with a deterministic, multi-stage engineering substrate.

Rather than installing disjointed standalone skills from assorted portals, Document OS packages a master format router, format-specific procedural protocols, safety rules, and battle-tested local Python CLI utilities into a unified, portable architecture.

The 5-Stage Document Lifecycle

Stage	Action	Tool / Script	Verification Objective
1. Triage	Preflight structure analysis	inspect_doc.py	Identify MIME, digital vs. scanned layers, sheets, formulas
2. Route	Format protocol selection	document-router	Select optimal toolchain (PDF, DOCX, XLSX, PPTX, OCR)
3. Execute	Deterministic processing	extract / convert_doc	Execute Python library or headless LibreOffice/Pandoc engine
4. Render	Visual verification preview	render_doc.py	Generate high-resolution PNG previews for multi-modal review
5. QA	Structural integrity check	qa_doc.py	Scan formula errors (#REF!), validate OOXML tree, confirm pa

2. Deployment & Usage Models

Document OS is designed for dual deployment: it can run isolated within a specific project repository or installed globally across your machine for every Antigravity project.

A. Workspace Deployment (.agents/)

When stored in .agents/plugins/document-os/ and accompanied by a root GEMINI.md, the agent automatically discovers the plugin and rules upon launching within the directory. This enables team collaboration by checking Document OS directly into your project's Git repository.

Command line invocation example:

```
python .agents/plugins/document-os/scripts/inspect_doc.py financial_model.xlsx
```

B. Global Deployment (~/.gemini/config/)

To make Document OS available to *any* folder or repository opened in AntigraVity, deploy it to your global configuration directory. Run the automated PowerShell installer with the global deployment flag:

```
.\install_document_os.ps1 -DeployGlobal
```

This copies the plugin to \$HOME\.gemini\config\plugins\document-os\, making the skills and rules universally available across every agent session on your workstation.

3. Format Engineering Protocols

Portable Document Format (PDF)

- **Digital PDFs:** Extract structural text streams with pypdf. Extract grid tables with pdfplumber.
- **Scanned PDFs:** Render pages to 300 DPI raster images, apply Otsu contrast thresholding, and run Tesseract OCR.
- **Generation:** Use programmatic Flowables via ReportLab (avoid manual string concatenation).

Microsoft Word (DOCX)

- **Semantic Styles:** Modify runs and paragraphs using Word styles rather than ad-hoc inline font styling.
- **Structural Preservation:** Retain section properties (headers, footers, margins) during in-place edits.
- **Conversions:** Headless LibreOffice provides 100% vector-faithful DOCX → PDF rendering.

Microsoft Excel (XLSX)

- **The Cardinal Rule:** Never clobber live calculation formulas (e.g. =SUM()) with static scalar values.
- **Multi-Sheet Integrity:** Inspect all sheets before queries. Check formula graphs for downstream dependencies.
- **Automated Formula Error Scan:** qa_doc.py scans for #REF!, #VALUE!, and #DIV/0! tokens.

Microsoft PowerPoint (PPTX)

- **Slide Master Layouts:** Inherit from standard master slide archetypes (16:9 widescreen default).
- **Overflow Defense:** Enforce bounding box constraints and word wrap to prevent clipped bullet points.
- **Visual QA:** Automatically export slides to PDF and render to PNGs for vision-model layout validation.

4. Cross-Harness Portability

Document OS strictly follows the open **Agent Skills Standard** (YAML frontmatter + progressive markdown). The underlying Python CLI utilities are 100% standalone and platform-agnostic. Use the cross-harness exporter:

```
.\export_cross_harness.ps1 -Target ClaudeCode
```

```
.\export_cross_harness.ps1 -Target OpenCode
```

Agent Harness	Standard Customization Path	Integration Method
Google Antigravity	.agents/plugins/ or ~/.gemini/config/plugins/	Native Plugin discovery & progressive disclosure
Claude Code	.claude/skills/ or ~/.claude/skills/	Agent Skills directory export
OpenCode CLI / Cursor	.skills/ or ~/.config/opencode/skills/	Universal symlink & standalone CLI tool access

5. CLI Helper Command Cheat Sheet

Script	Key Arguments	Purpose
inspect_doc.py	<file>	Returns structured JSON metadata, page counts, scanned vs text
extract_doc.py	<file> --type text tables images	Extracts raw text, Markdown tables, or embedded images
convert_doc.py	<input> <output_or_ext>	Headless conversion via LibreOffice, Pandoc, or pdf2docx
render_doc.py	<file> --outdir <dir>	Renders PDF pages / Office slides to high-res PNGs
ocr_doc.py	<file> --lang eng --psm 3	Runs Tesseract OCR with adaptive contrast enhancement
qa_doc.py	<file>	Validates structural integrity, formula syntax, and exit codes